Lab 7: Fuzzing for Input Validation Bugs

Name: Vasu Tripathi
SRN: PES1UG24AM918
Section: G
Repository: https://github.com/Vasu-Tripathi/L6_L7_39_password_policy

---

Objective

Use property-based fuzz testing with Hypothesis to find input-validation bugs in password_policy.py, then harden the code so it handles every unexpected input safely.

Module Under Test

- `first_character_type(pw)` – classifies the first character as digit, letter or symbol.
- `split_credentials(text)` – splits "user:password" into a (user, password) tuple.
- `digit_ratio(pw)` – fraction of characters that are digits.

Step 2: Running the Buggy Application

```
$ python password_policy.py
```

```
vasut@Vasu MINGW64 ~/Documents/SE LAB 6/L6_L7_39_password_policy/lab6 (main)
$ cd ../lab7
$ python password_policy.py
==== Running first_character_type ====
Enter a password:
Traceback (most recent call last):
  File "C:\Users\vasut\Documents\SE LAB 6\L6_L7_39_password_policy\lab7\password_policy.py", line 33, in <module>
    main()
  File "C:\Users\vasut\Documents\SE LAB 6\L6_L7_39_password_policy\lab7\password_policy.py", line 19, in main
    result = first_character_type(user_input)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\vasut\Documents\SE LAB 6\L6_L7_39_password_policy\lab7\password_policy.py", line 2, in first_character_type
    ch = pw[0]
         ~~^^^
IndexError: string index out of range
```

Figure 1: Original program crashes with IndexError when an empty password is entered

Step 3–4: Fuzz Tests on the Buggy Code

Test file: `test_password_policy.py`. Each function has a "no crash" test fed with `st.text() | st.none()`, plus property tests: digit_ratio always returns a float in [0, 1], and split_credentials preserves colons inside the password.

```
$ python -m pytest test_password_policy.py
```

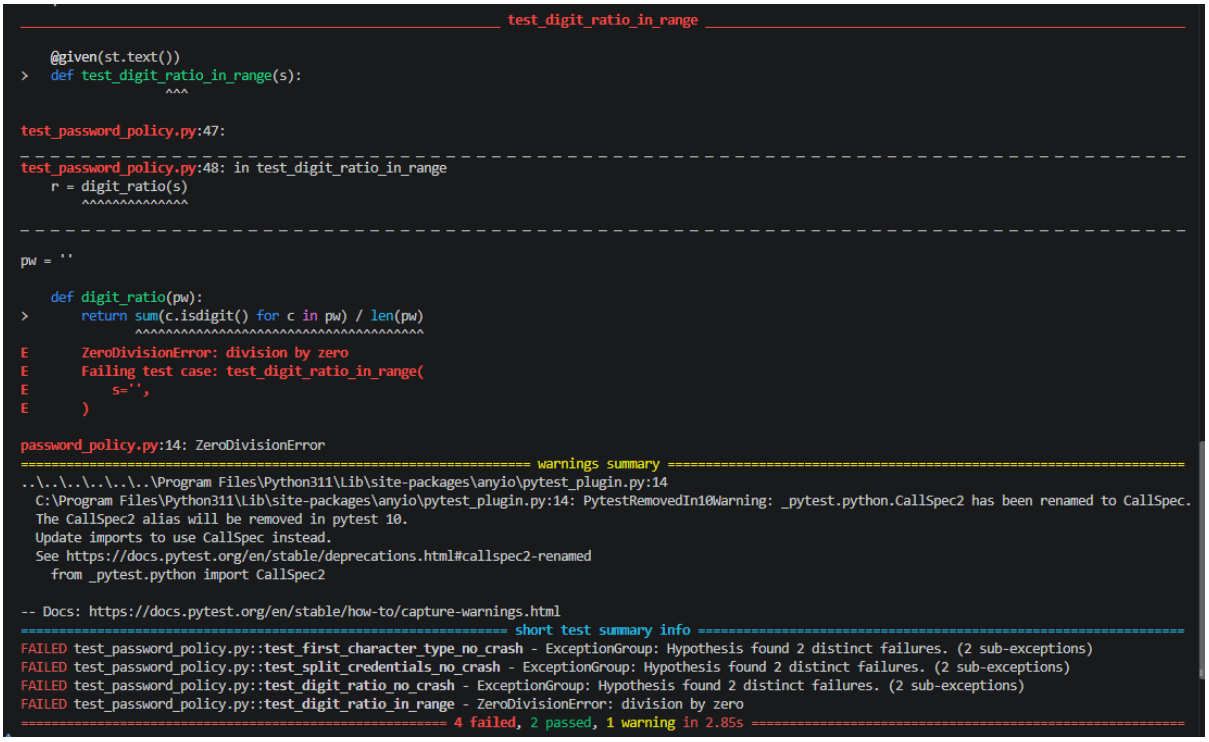


Figure 2: Fuzz tests on the buggy code – 4 failed, 2 passed

Bugs Found by Hypothesis

Function	Input	Error	Cause
first_character_type	''	IndexError	pw[0] on empty string
first_character_type	None	TypeError	None is not subscriptable
split_credentials	'' / no ':'	ValueError	split gives 1 value, 2 expected
split_credentials	None	AttributeError	None has no .split()
split_credentials	'a:b:c'	ValueError	Extra ":" gives too many values
digit_ratio	''	ZeroDivisionError	len(pw) is 0
digit_ratio	None	TypeError	None is not iterable

Hypothesis shrank every failure to the smallest example (an empty string or None). Because inputs are random, the colon-in-password property did not trigger on this run, but the bug was reproduced separately with "bob:pa:ss".

Step 5: Fixes Applied

- **Type check:** every function first checks isinstance(x, str) and returns a safe value for non-strings.
- **first_character_type:** returns "invalid" for non-strings and "empty" for "".
- **split_credentials:** returns (None, None) when the input is not a string or has no ":"; uses split(":", 1) so passwords may contain ":".
- **digit_ratio:** returns 0.0 for non-strings and empty strings, avoiding division by zero.

- **main():** prints "Invalid format" instead of crashing when the credentials are malformed.

```
$ python password_policy.py
```

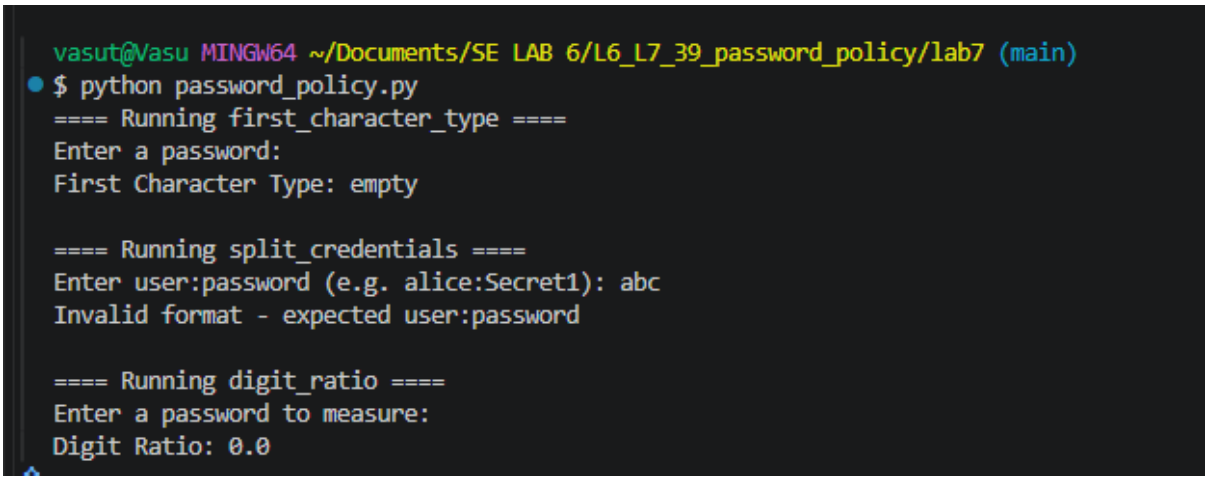


Figure 3: Fixed program handles empty input and a missing colon without crashing

Step 6: Fuzz Tests on the Fixed Code

```
$ python -m pytest test_password_policy.py
```

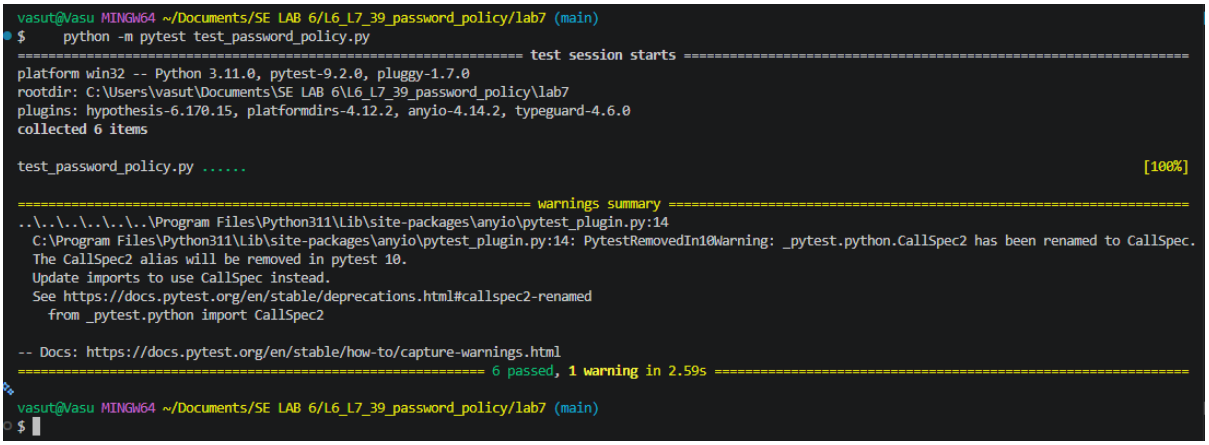


Figure 4: All fuzz tests pass on the fixed code – 6 passed

Conclusion

The hand-written example inputs never exercised empty strings, None, or extra delimiters, but Hypothesis found all of these within seconds. Adding type checks, empty-input guards and a bounded split made all three functions robust, and every property now holds for thousands of generated inputs.